

Part A: mid-session passphrase check-in

Build spec for Part A, starting from an empty repo.

Scope

- Build and set up everything from scratch. One command gives a working local stack.
- Three fields: verified email, timestamp, session passphrase.
- Zoom hides guest IDs and emails (since March 2023), so it can't prove attendance.
- A Verified-email Google Form opens 15 to 25 minutes in and asks for a passphrase the teacher says and shows.
- Staff use a web console; the system provisions, publishes and pulls forms. CU has no data manager.

Design invariants

1. **Never drop a submission.** Wrong passphrase, unknown email, out of window: record it with the reason.
2. `checkin` (observation) is immutable. `attendance_decision` (judgment) is versioned: supersede, never mutate.
3. Every decision records rule, model, human, and when.
4. A human override always wins; no automated pass overwrites it.
5. Ingest is idempotent: a rerun writes zero rows.
6. Identity never blocks ingest; unknown emails go to a review queue.
7. Read back external state (published, Verified). A 200 proves nothing.
8. Everything is cohort-keyed. Timestamps are UTC past the parser.

Google API traps

Current as of August 2026. All fail silently.

- Trap 1: API forms created after June 30, 2026 start unpublished and refuse responses.
 - Call `forms.setPublishSettings()` with `publishState: { isPublished: true, isAcceptingResponses: true }`, then read back and assert.
- Trap 2: `emailCollectionType: VERIFIED` via `batchUpdate` → `updateSettings` can fail with `400 INVALID_ARGUMENT`. Use template-and-copy:
 - The app creates one template; a human sets Settings → Responses → Collect email addresses → Verified (once).
 - The app reads `form.settings` back and blocks until Verified, and re-checks before each provisioning run.
 - Session forms are Drive `files.copy()` of the template (settings carry over). `batchUpdate` sets only title, description, question text.
- Trap 3: REST can't link a response Sheet (`Form.setDestination()` is Apps Script only).
 - Read `forms.responses.list` (`respondentEmail`, RFC3339 UTC), polling with `filter=timestamp > N` and a per-form watermark.
 - CSV import stays as a fallback.

- Trap 4: service accounts can't own Forms. Use user OAuth, not domain-wide delegation.
 - Scopes: `https://www.googleapis.com/auth/forms.body` and `https://www.googleapis.com/auth/drive.file` only.
 - Refresh token encrypted in Postgres, never in a file. Forms stay in CU's Drive.

Deliverable 1: Session console

- FastAPI + Jinja2 + HTMX (Next.js on Supabase is the alternative; don't build both).
- Login: Google sign-in against a config allowlist of CU emails. No passwords.

Screens:

1. Connect Google: status, account, scopes, disconnect.
2. Template setup: create, show link and Verified step, "Verify template". Green only on `VERIFIED`; blocks everything after.
3. Sessions: list, create, edit.
4. Session detail: "Provision form" (copy, set title and question, publish, verify, store ID and URL); URL with copy button and QR code; "Announce now" stamps `announced_at_utc`; live count; "Pull responses".
5. Review: `needs_review` queue with one-click attended / not attended; tab of AI decisions with reasoning.

Session fields:

- Required: `title`, `scheduled_at`, `timezone` (IANA, browser default), `duration_minutes`, `cohort_id`.
- Optional: `grace_minutes` (default 15, both sides), `passphrase` (absent is legal), `announced_at`.

Passphrase and form:

- Inline guidance: one word, 5 to 10 letters, no homophones, nothing from slides, never reused. "Suggest a passphrase" uses a homophone-free wordlist.
- Warn before saving on reuse within the cohort.
- UI: say it aloud and show it (for deaf fellows); it's one signal, never proof alone.
- Provisioning is idempotent per session, logs every attempt, supports `--dry-run`.
- One required short answer, "Today's passphrase". No session dropdown.
- Plain header: what's collected, who sees it, why. Leave `TODO(retention)`; **don't invent a period.**

Deliverable 2: Supabase (Postgres)

- Supabase CLI, local only. Run `supabase init`, write migrations (`supabase/migrations/`) and seed (`supabase/seed.sql`), verify `supabase start + supabase db reset`.
- Postgres `postgres://postgres:postgres@localhost:64322/postgres`; Studio `http://localhost:64323`.
- Needs Docker; fail clearly if unreachable. `psycopg 3` and plain SQL, not `supabase-py`.
- Types: `uuid`, `timestampz`, `jsonb`, `text` with CHECK.

Schema:

- `cohort`: `cohort_id` PK, label, start/end dates.

- `fellow`: `fellow_id` PK (CU-issued), `cohort_id` FK, `full_name`, `primary_email`, `status`.
- `session`: console fields plus `scheduled_at_utc`, `announced_at_utc`.
- `google_credential`: email, encrypted refresh token (key from env), scopes, `connected_at`, `last_refreshed_at`.
- `form_template`: form ID, `verified_email_confirmed_at`, settings snapshot, `last_verified_at`.
- `session_form`: `session_id` FK, `form_id`, `form_url`, `provisioned_at`, `published_at`, `publish_verified_at`, `response_watermark`, `last_polled_at`.
- `checkin`: `checkin_id` uuid PK, `source_event_id` UNIQUE, `source` (`forms_api|csv`), `submitted_email`, `submitted_at_utc`, `submitted_at_raw`, `source_timezone` (NULL on API), `session_id` FK null, `session_match` (`matched|none|ambiguous`), `passphrase_raw`, `passphrase_match`, `edit_distance`, `latency_seconds`, `extra_fields` jsonb, `load_id` FK, `ingested_at`.
 - Stores email, not `fellow_id`; roster join at read time means no backfill.
- `attendance_decision`: `decision_id` uuid PK, `checkin_id` FK, `attended` bool null, `status` (`attended|not_attended|needs_review`), `confidence` 0 to 1, `decided_by` (`rule|ai|human`), nullable `rule_name`, `ai_model`, `ai_prompt_version`, `ai_reasoning`, `human_email`, `note`, `superseded_at` (NULL = current), `created_at`.
 - Partial unique index on `checkin_id` WHERE `superseded_at` IS NULL. Supersede = UPDATE + INSERT.
- `ai_adjudication_cache`: key (`expected_normalized`, `submitted_normalized`, `prompt_version`, `model`); `verdict`, `confidence`, `reasoning`, `created_at`.
- `identity_unresolved`: email, first/last seen, count, optional best guess and score.
- `load_run`: source, origin, input SHA-256, start/finish, rows read/written/skipped, status, error.
- `provisioning_log`: session, action, request summary, outcome, error, at.
- RLS on `fellow`, `checkin`, `attendance_decision`, `google_credential`; pipeline uses the service role. Leave a `TODO(access)` stub. **Don't invent the policy.**

Deliverable 3: Response ingestion

- API: `forms.responses.list` with `filter=timestamp > <watermark>`, page via `nextPageToken`, advance the watermark only after a full successful loop.
- CSV: `--sheet-timezone` mandatory, no default; missing fails naming the flag. Convert with `zoneinfo` (DST-safe), store raw string and zone. Headers any case and order; unknown columns into `extra_fields`.
- `source_event_id` = SHA-256 of (`form_id_or_file`, `normalized_email`, `submitted_at_utc_iso`), stable across both paths. UNIQUE + ON CONFLICT DO NOTHING.
- Window [`scheduled_at_utc - grace`, `scheduled_at_utc + duration + grace`]: one `matched`, none `none`, several `ambiguous` (warn naming them). Unmatched rows get NULL `session_id` and are still written.
- API path: a timestamp outside its form's session window is a config error to flag, not reassign.
- Emails: strip, lowercase; keep Gmail dots and `+suffix`. Match `fellow.primary_email` in cohort; misses upsert `identity_unresolved`. No fuzzy name linking.

Deliverable 4: Tier 1 rules

- Normalize: trim, collapse whitespace, lowercase, strip punctuation. Hand-write Levenshtein.
- Outcomes: `exact`, `fuzzy` (`distance ≤ max_edit_distance`, default 1, on by default), `mismatch`, `not_set`, `no_session`.

- Decisions:
 - `exact` in window: `attended`, `exact_match`, 1.0
 - `fuzzy` in window: `attended`, `fuzzy_match`, 0.9
 - `not_set` in window: `attended`, `no_passphrase_required`, 0.7
 - `no_session`: `not_attended`, `outside_all_windows`, 0.6
 - `mismatch` in window: tier 2

Deliverable 5: Tier 2, Gemini

- `google-genai` (from `google import genai`; `client = genai.Client()`), not deprecated `google-generativeai`.
- `gemini-2.5-flash` default (free: 10 RPM / 250 RPD; Flash-Lite 15 RPM / 1,000 RPD). `GEMINI_API_KEY` in gitignored `.env`; commit `.env.example`.
- Send only expected and submitted strings. Schema: `{ "heard_the_passphrase": true, "confidence": 0.0, "reasoning": "one sentence" }`.
- `temperature=0`; store `PROMPT_VERSION = "v1"` on each decision. Check `ai_adjudication_cache` first, write through.
- Backoff on 429; config cap per run. `--no-ai` skips.
- Unavailable: `status='needs_review'`, `decided_by='rule'`, `rule_name='ai_unavailable'`; the pipeline still finishes.
- `needs_review` **never becomes** `not_attended`.

Deliverable 6: Tier 3, human override

- `cufa decide` or console: supersede, insert `decided_by='human'`, `confidence=1.0`, email, note.
- Adjudication skips human decisions; `--force` overrides with a loud warning naming what it overwrites.
- `cufa review --status needs_review` (oldest first) and `--status ai` (for spot-checks).

Deliverable 7: Latency

- Seconds from `session.announced_at_utc`, else from earliest matched submission (first gets 0; document it). NULL with no session.
- Store only: no thresholds or flags.

Deliverable 8: CLI

```
cufa db up | down
cufa serve
cufa google connect | status
cufa template create | verify
cufa load-roster      --csv <path> --cohort <id>
cufa load-sessions   --csv <path>
cufa provision        --session <id> [--dry-run]
cufa pull             --session <id> | --cohort <id>
cufa ingest part-a    --csv <path> --cohort <id> --sheet-timezone <IANA>
cufa adjudicate       --cohort <id> [--no-ai] [--force]
cufa decide           --checkin <id> --status <s> --by <email> --note "<text>"
cufa review           [--status needs_review|ai|unresolved-identity]
cufa report           --cohort <id> [--json]
```

- Everything in the console must also work from the CLI.

Deliverable 9: Test harness

- No Google account or Gemini key needed.
- `make setup` (deps, supabase init, Docker check), `make demo` (reset, fixtures, load, ingest via fake client, `--no-ai`, report), `make demo-ai` (skips without key), `make demo-console` (all screens, fake client), `make test` (offline), `make clean` (supabase stop, delete fixtures).
- Document SQL for: decisions per fellow, review queue, AI reasoning, cache hit rate, provisioning log.
- `FakeGoogleClient`: `create_template`, `read_settings`, `copy_form`, `batch_update`, `set_publish_settings`, `list_responses`. Simulates unpublished form, 400, `RESPONDER_INPUT` template, pagination, 429 then success.
- `scripts/generate_fixtures.py` (fixed seed): 20 fake fellows at `@example.invalid`, 6 sessions (one without passphrase), and responses covering every edge case in the tests plus conversational, wrong and blank answers.

Deliverable 10: Tests

Offline, fake Google and Gemini, no real fellow data.

- Ingest: rerun skips all; API then CSV doesn't duplicate; reorder keeps IDs; `2026-09-15 13:05:00 America/New_York` → `2026-09-15T17:05:00Z`; both DST sides; missing `--sheet-timezone` errors; all three match states write; unknown email → `identity_unresolved`; `a.b@gmail.com` ≠ `ab@gmail.com`; extra column kept.
- Google: unpublished read-back raises; publish always called; `RESPONDER_INPUT` blocks provisioning; 400 leaves nothing marked ready; all pages read; watermark holds on mid-pull failure; no second form on re-provision.
- Adjudication: five outcomes; `" Justice "`, `"JUSTICE"`, `"justice."` → `exact`; latency cases; index enforces one current decision; human survives rerun, `--force` warns; repeated pair calls once; no key → `needs_review`, never `not_attended`.
- Security: refresh token unreadable in raw DB; no email, token or key in INFO+ logs.

Deliverable 11: Documentation

- `docs/setup/console.md`, `docs/setup/google-cloud.md` (APIs, OAuth client, scopes), `docs/setup/local-dev.md` (Docker, CLI, Studio, make, SQL).
- `docs/google-api-traps.md`: the four traps and fixes, for the next maintainer.
- `docs/decisions.md` ADRs: form-over-Zoom-API, Verified-over-responder-input, template-copy-over-API-settings, responses-API-over-linked-Sheet, user-OAuth-over-service-account, timestamp-over-dropdown, never-drop-a-submission, observation-separate-from-decision, three-tier adjudication.
- `README.md`: clone to demo in under ten minutes.

Constraints

- Python 3.11+. Deps: `fastapi`, `uvicorn`, `jinja2`, `psycopg[binary]`, `google-genai`, `google-auth`, `google-auth-oauthlib`, `google-api-python-client`, `cryptography`, `python-dotenv`, `pytest`; `rest stdlib`.
- No committed secrets. Emails only at DEBUG; never log keys or tokens.
- Type-hint public functions; docstrings explain why.

Acceptance criteria

1. `make setup` && `make demo` prints a report on a clean machine, no Google account or `GEMINI_API_KEY`.
2. Rerunning `make demo` gives identical output.
3. `count(checkin) == count(fixture responses)`.
4. One current decision per check-in, enforced by the index.
5. A human override survives `adjudicate`.
6. `make demo-console` covers every screen, zero Google calls.
7. Fake provisioning fails loudly on unpublished forms or non-Verified templates.
8. `make demo-ai` sends only `mismatch` cases; a second run makes zero calls.
9. `make test` passes offline.
10. `docs/google-api-traps.md` is accurate.

Out of scope

- Part B fields; Slack or Zoom integration; auto-posting links, triggers, reminders.
- Gamification; cross-component scoring or at-risk flags; dashboards beyond review screen and report.
- Supabase Cloud or public hosting; domain-wide delegation.

See also

- `docs/implementation/part-b-exit-checkin.md`